Guía de Actividades Práctico-Experimentales Nro. 002

1. Datos Generales

Asignatura	Sistemas Distribuidos
Ciclo	6to
Unidad	Uno
Resultado de aprendizaje de la unidad	R2. Escribe un programa que realiza cualquier requerimiento de clasificación y conversión en unidades de mensajes, como paquetes, para comunicar información interesante entre dos hosts, bajo los principios de solidaridad, transparencia, responsabilidad y honestidad.
Práctica Nro.	2
Título de la Práctica	Arquitectura de Sistemas Distribuidos (comunicación cliente–servidor con archivos)
Nombre del Docente	José O. Guamán Q.
Fecha	06 de octubre de 2026 09 de octubre de 2026
Horario	Lunes 10h30 – 11h30 Viernes 07h30 – 09h30
Lugar	Aula
Tiempo planificado en el Sílabo	3 horas

2. Objetivo(s) de la Práctica:

Comprender la arquitectura de sistemas distribuidos mediante el desarrollo independiente de sus componentes (cliente y servidor).

3. Materiales y reactivos:

Guia practica

Se requiere un sistema que permita crear un servidor y un cliente con un canal de comunicación (archivo de texto), se debe evidenciar que se esta llenado el archivo.

4. Equipos y herramientas

Hardware

- Computador

Software

- Lenguaje de programación

- IDE de programación
- Archivo de texto

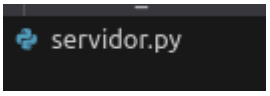
5. Procedimiento / Metodología

Parte 1: Desarrollo del SERVIDOR (1 hora 20 minutos)

Este módulo se desarrolla de forma independiente

Actividad

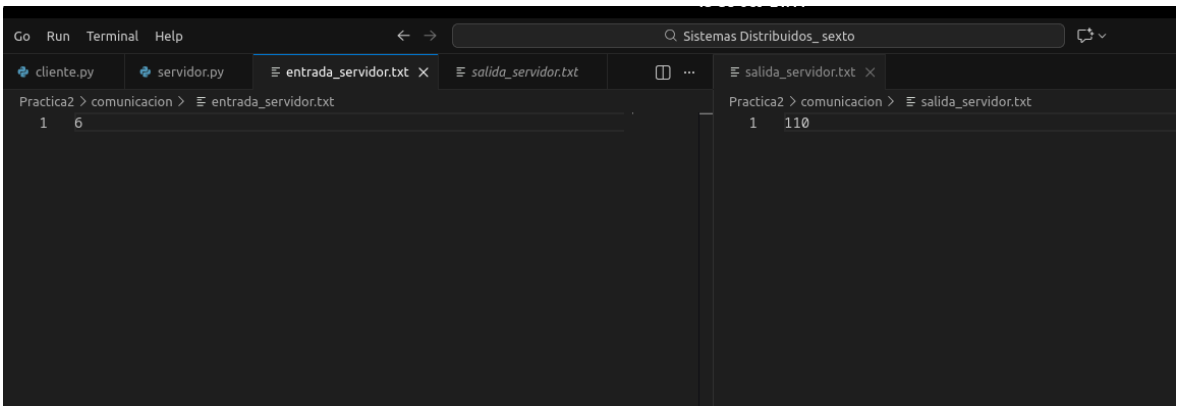
1. Crear un archivo: servidor.py



2. Crear un archivo auxiliar: entrada_servidor.txt



3. Implementar un programa que:
   - Revise periódicamente el archivo entrada_servidor.txt.
   - Detecte si existe contenido nuevo.
   - Procese el mensaje:
     - Ejemplo: convertir texto a mayúsculas o invertirlo.
   - Escriba el resultado en: salida_servidor.txt



4. Condiciones:
   - Debe ejecutarse continuamente (simulación de servicio activo).
   - No debe reprocesar el mismo mensaje.
   - Debe manejar archivos vacíos.

🔷 **Parte 2: Desarrollo del CLIENTE (1 hora)**

Se desarrolla sin necesidad de ver el código del servidor

Actividad

1. Crear archivo: cliente.py

cliente.py

2. Implementar un programa que:
    - Solicite un mensaje al usuario.

```
~/Sistemas Distribuidos_ sexto/Practica2
❯ python cliente/cliente.py
Cliente listo. Deja el servidor ejecutándose en otra terminal.
Escribe 'salir' para terminar.

Mensaje: 56
Respuesta: 111000

Mensaje: 24
Respuesta: 11000

Mensaje: 7854
Respuesta: 1111010101110

Mensaje: 9688
Respuesta: 10010111011000

Mensaje:
```

    - Escriba el mensaje

en: entrada_servidor.txt

cliente.py | servidor.py | entrada_servidor.txt | salida_servidor.txt

Practica2 > comunicacion > entrada_servidor.txt

```
1  56
```

    - Espere un tiempo (simulando latencia).
    - Lea la respuesta

desde: salida_servidor.txt

Practica2 > comunicacion > salida_servidor.txt

```
1  111000
```

    - Muestre el resultado en pantalla.
3. Condiciones:
    - No debe asumir cómo funciona el servidor internamente.
    - Solo debe conocer los archivos de entrada y salida.

🔹 Parte 3: Integración del sistema (25 minutos)

1. Ejecutar primero el servidor.
2. Ejecutar el cliente.
3. Enviar varios mensajes.
4. Verificar:

- Flujo correcto
- Respuesta adecuada

NOTA: los archivos de comunicación deben estar en una sola carpeta fuera de ambos proyectos

6. Resultados esperados:

Al finalizar la práctica, el estudiante entregará:

- Código fuente de servidor.py y cliente.py (repositorio Git).
- Capturas de pantalla que evidencien: (a) el archivo entrada_servidor.txt siendo llenado por el cliente, (b) salida_servidor.txt con el resultado procesado, (c) al menos 3 mensajes procesados correctamente.
- Respuestas a las preguntas de control en el reporte de práctica. Con ello demostrará: el desacoplamiento entre cliente y servidor, la comunicación indirecta por medio compartido, y las limitaciones del modelo (tiempo real, conflictos de escritura, dependencia del medio).

7. Preguntas de Control:

¿Qué significa que cliente y servidor estén desacoplados?

Significa que cada programa tiene una función propia y puede ejecutarse por separado. El cliente no necesita conocer los detalles internos del servidor: escribe una solicitud y espera una respuesta. El servidor lee la solicitud, la procesa y publica el resultado.

¿Qué ventajas tiene desarrollar componentes por separado?

Permite dividir responsabilidades: el cliente se ocupa de interactuar con el usuario y el servidor de procesar el dato. Así, cada parte puede desarrollarse, probarse o modificarse con más independencia, siempre que ambas respeten el formato y el mecanismo de comunicación acordados.

¿Qué problemas pueden surgir al integrar sistemas independientes?

Pueden aparecer incompatibilidades en el formato de los mensajes, errores al leer o escribir los archivos, diferencias en las rutas de acceso y problemas de sincronización. Por ejemplo, el cliente podría leer una respuesta anterior o el servidor podría procesar una solicitud mientras el cliente ya la está reemplazando.

¿Qué sucede si el servidor no está ejecutándose?

El cliente escribe la solicitud, pero nadie la procesa ni genera una respuesta. Por eso, espera hasta el límite de 30 segundos y luego muestra un error indicando que no llegó respuesta y que se compruebe si el servidor está activo.

¿Cómo se simula la comunicación en esta práctica?

Mediante archivos compartidos, no mediante una conexión de red. El cliente escribe el número en entrada_servidor.txt y consulta periódicamente salida_servidor.txt; el servidor revisa la entrada cada segundo, convierte el número y escribe el resultado en la salida. Por ejemplo, 9688 produce 10010111011000.

¿Qué limitaciones tiene este modelo frente a uno real?

Requiere que ambos programas puedan acceder a los mismos archivos y está pensado para una comunicación sencilla. La consulta periódica

introduce demoras, los archivos solo guardan una solicitud y una respuesta a la vez, y no hay mecanismos robustos para identificar mensajes, manejar solicitudes simultáneas o recuperarse de fallos como los que suelen incluir los sistemas reales de red.

¿Cómo se podría mejorar la sincronización?

Se podrían añadir identificadores de solicitud para relacionar cada respuesta con su petición, usar bloqueos o una cola para evitar que varias escrituras se pisen y establecer tiempos de espera y confirmaciones. Para una comunicación real, se podría reemplazar la consulta de archivos por sockets o por una cola de mensajes.

¿Qué pasaría si múltiples clientes usan el sistema?

Como todos usarían los mismos dos archivos, sus solicitudes podrían sobrescribirse entre sí y un cliente podría recibir la respuesta destinada a otro. Para soportar varios clientes se necesitarían solicitudes y respuestas identificadas individualmente, una cola de mensajes o un servidor que gestione varias conexiones concurrentes.

8. Evaluación

NO APLICA

9. Bibliografía

Semana 2. Guaman, Jose. 2026

10. Elaboración y Aprobación

Elaborado por	José Guamán **Docente**	Validar únicamente en F Firmado electrónicamente JOSE OSWALDO GUAMAN QUINCHE
Ejecutado Por	Rene Rivera **Técnico Docente**	Validar únicamente en F Firmado electrónicamente RAMIRO RENE RIVERA GUAMAN
Aprobado por	Edison L Coronel Romero **Director de Carrera**	Firmado electrónicamente por: EDISON LEONARDO CORONEL ROMER Validar únicamente con FirmaEC